

C73-A5

Changze Wu Shuang Wu

October 2024

1 Q1

1.1 a

SUBPROBLEMS TO SOLVE : For each i , $1 \leq i \leq n$, let $\text{cos}[i]$ be the minimum of max empty space after placing the i books, which is the smallest possible maximum empty space left on any shelf when the i books are arranged.

Solving the original problem: The answer to our problem is simply finding the $\text{cos}[n]$.

Recursive formula to compute the subproblems:

$$\text{cos}[i] = \min_{0 < j \leq i} (\max(\text{cos}[j], L - (d[j+1] + d[j+2] + \dots + d[i])))$$

Justification why (†) correctly computes (*): There are two cases:

- case 1: If $j = i$, the shelf only contains book i . The empty space on this shelf is $L - d[i]$. The formula evaluates: $\text{cos}[i] = \max(\text{cos}[i-1], L - d[i])$. This configuration considers the maximum of the previous configuration ($\text{cos}[i-1]$) and the empty space remaining when the i -th book is placed alone on the new shelf - case 2: If $0 < j < i$, then the formula evaluates placing books $j+1$ through i together on the last shelf and combines this configuration with the optimal configuration up to j . Specifically: $\text{cos}[i] = \min_{0 < j \leq i} \left(\max(\text{cos}[j], L - \sum_{k=j+1}^i d[k]) \right)$. The formula combines this value with $\text{cos}[j]$, the minimum maximum empty space up to j . The maximum of these two values represents the worst-case empty space when placing books up to i . In the meanwhile, formula takes the minimum of the maximum empty spaces for all possible values of j . This ensures that the configuration chosen for $\text{cos}[i]$ is the one that results in the smallest possible maximum empty space. which **QED**

Pseudocode:

Running time: for line 1 and 2, there is $\Theta(1)$. line 4 is $\Theta(n)$ and line 6 is $\Theta(n)$. The others are all constant time. So the final running time is $\Theta(n^2)$

Correctness : we prove by induction:

base case : $i = 1$, then there is only book on the first shelf, if the d_1 is greater than L , then it is feasible. Otherwise, we get what we want, we are done.

Algorithm 1 $\text{DO}(\text{book}, L)$

```
1:  $n := \text{length}(\text{book})$ 
2: for  $i := 1$  to  $n$  do
3:    $\text{cos}[i] := \infty$ 
4: end for
5: for  $i := 1$  to  $n$  do
6:    $\text{sum} := 0$ 
7:   for  $j := i$  to  $1$  do
8:      $\text{sum} += d_j$ 
9:     if  $\text{sum} > L$  then
10:      break
11:    end if
12:     $\text{cos}[i] = \min(\text{cos}[i], \max(\text{cos}[j-1], L-\text{sum}))$ 
13:  end for
14: end for
15: return  $\text{cos}[n]$ 
```

IH : Assume that the algorithm correctly computes $\text{cos}[j]$ for all $j \leq i$. We want to prove that it also computes $\text{cos}[i]$ correctly. When we consider placing first i books:

a) The algorithm computes $\text{cos}[i]$ by considering different ways to place the last segment of books on the current shelf, specifically books j to i , for $j \leq i$.

b) For each j , the algorithm calculates sum , the total width of books from j to i . If $\text{sum} \leq L$, this segment is feasible to place on the current shelf.

c) the algorithm will run the line 11 which a) $\text{cos}[j-1]$ represents the minimum maximum empty space when the first $j-1$ books are optimally placed. and b) $L - \text{sum}$ is the empty space remaining on the current shelf if books j through i are placed on it.

Thus, if the algorithm is correct for all $j \leq i$, it will also be correct for i . By induction, the algorithm is correct for all i from 1 to n . QED

1.2 b

: - Same as question a: $\text{cos}[i]$ has the same definitions.
- here we define *split* as an array of size n to store the split point for each i .
- $\text{split}[i]$: This array stores the point where the split occurred when computing $\text{cos}[i]$. It helps us trace back and determine the books that go on each shelf. After filling the cos and split arrays, we start at the last book ($i = n$) and move backward using $\text{split}[i]$ to identify the range of books that go on each shelf.

Pseudocode:

Algorithm 2 DORETROFIT(book, L)

```

1:  $n := \text{length}(\text{book})$ 
2: for  $i := 1$  to  $n$  do
3:    $\text{cos}[i] := \infty$ 
4:    $\text{split}[i] := -1$ 
5: end for
6: for  $i := 1$  to  $n$  do
7:    $\text{sum} := 0$ 
8:   for  $j := i$  to  $1$  do
9:      $\text{sum} += d_j$ 
10:    if  $\text{sum} > L$  then
11:      break
12:    end if
13:     $\text{currentcost} = \max(\text{cos}[j-1], L - \text{sum})$ 
14:    if  $\text{currentcost} < \text{cos}[i]$  then
15:       $\text{cos}[i] := \text{currentcost}$ 
16:       $\text{split}[i] := j-1$ 
17:    end if
18:  end for
19: end for
20:  $\text{place} = []$ 
21:  $i := n$ 
22: while  $i > 0$  do
23:    $j = \text{split}[i]$ 
24:    $\text{place.insert}$  (add the range of books for the current shelf)
25:    $i = j$ 
26: end while
27: return  $\text{cos}[n], \text{place}$ 

```

Correctness: To prove the algo, we need to prove that 1) The optimal solution to the problem can be constructed from optimal solutions to its subproblems. and 2) The algorithm correctly identifies and computes the minimum possible maximum empty space using the subproblems.

for 1) The critical observation is that, for any optimal placement of the first i books, there must be an optimal way to place the first j books (for some j

$< i$), and the remaining books ($j+1$ through i) must be placed on the last shelf. Therefore, the problem for $\text{cos}[i]$ depends on finding the best split point j that minimizes the maximum empty space. In the meanwhile, The algorithm evaluates each possible split j (from 0 to $i-1$), ensuring that the solution for $\text{cos}[i]$ is composed of the optimal solution for the first j books (stored in $\text{cos}[j]$) combined with the remaining books on the last shelf. This ensures that the optimal solution to the larger problem is built upon the optimal solutions to its smaller subproblems. As we mentioned in a part

For 2) The condition $\text{sum}(d_{j+1} \dots d_i) \leq L$ ensures that the books from $j+1$ to i fit on the last shelf. This ensures the placement is feasible. The expression $\max(\text{cos}[j], L - \text{sum}(d_{j+1} \dots d_i))$ calculates the maximum empty space when the first j books are placed optimally (as given by $\text{cos}[j]$), and the remaining books ($j+1$ through i) are placed on the last shelf. This correctly measures the maximum empty space across all shelves used up to the i -th book.

For each $i > 0$, the algorithm correctly finds the minimum maximum empty space by considering all valid ways to partition the books and combining the optimal subsolutions (i.e., the placement of the first j books and the placement of the remaining books on the last shelf). Therefore, if $\text{cos}[1]$ through $\text{cos}[i-1]$ are correctly computed, $\text{cos}[i]$ will also be correct. QED

Running time According to part a, The additional split array and the backtracking process take $O(n)$ time and space, which doesn't increase the overall time complexity of the algorithm beyond $O(n^2)$. Thus, the running time remains the same up to a constant factor.

2 Q2

subproblem to solve For each i , $0 \leq i \leq n$, let $dp[i]$ = number of division of $x[1..i]$.

solving the original problem : The answer to our problem is simply to find the $dp[n]$

Recursive formula to compute the subproblems :

$$dp[i] = \begin{cases} 1, & i = 0 \\ \sum_{j=1}^i (\text{Dict}(x, j, i) \times dp[j-1]) & \end{cases}$$

Justification why (†) correctly computes (*):

- case 1: $i=0$. when $i=0$, the string is empty according to the def, we can easily get the $dp[i] = 1$. then we are done
- case 2: $i \geq 0$. in this way we also need to consider some cases
- case 2a: no valid end at i . the dict func will return 0 and make the sum = 0.

- case 2b: at least one valid word ending at i . then the dict func will return 1. the term $dp[j-1]$ represents the number of ways to partition the prefix $x[1 \dots j-1]$. By multiplying dict by $dp[j-1]$, the formula correctly counts each partition of the prefix $x[1 \dots j-1]$ combined with the valid suffix $x[j \dots i]$.

Now, assume for contradiction that $dp[i]$ does not correctly compute the number of legal divisions of $x[1..i]$, which 1) means an additional legal division of $x[1..i]$ not accounted for in the formula, or 2) formula counts fewer legal divisions than actually exist.

for 1), here is a way to partition $x[1..i]$ that ends with a valid word $x[j..i]$, but this partition is not included in the sum. However, the formula explicitly sums over all j where $\text{dict} = 1$. Therefore, every valid word ending at i is considered, and each corresponding prefix partition count $dp[j-1]$ is included. Thus, no additional division could exist outside of what the formula captures.

for 2), here exists a valid partition of $x[1..i]$ where the suffix $x[j..i]$ is a valid word, but this partition is not correctly counted. However, if $x[j..i]$ is valid, then the formula includes $dp[j-1]$ in the sum, representing all ways to partition $x[1..j-1]$. Therefore, all such valid partitions are counted, and no fewer divisions than actually exist are possible.

conclude above all, the assumption lead to the contradiction

Pseudocode:

Running time : line 2 - 4 cost $\Theta(n)$ and line 5 - 10 cost $\Theta(n^2)$, line 12 return $\Theta(1)$. Thus, the total running time $\Theta(n^2)$

Algorithm 3 $DO(x)$

```
1:  $n := \text{length}(x)$ 
2: for  $i := 1$  to  $n$  do
3:    $dp[i] := 0$ 
4: end for
5: for  $i := 1$  to  $n$  do
6:   for  $j := 1$  to  $i$  do
7:     if  $\text{Dict}(x, j, i) == 1$  then
8:        $dp[i] += dp[j-1]$ 
9:     end if
10:  end for
11: end for
12: return  $dp[n]$ 
```

3 Q3

subproblem to solve For any i , $1 \leq i \leq n$, let $dp_T[i]$ be the maximum profit up to day i which Emon can get if day i he is in Toronto . $dp_M[i]$ indicate the max profit get in Melbourne same situation.

solving the original problem : The solution to the original problem, which is the maximum profit Emon can earn over the entire n days $\text{max_profit} = \max(dp_T[n], dp_M[n])$

Recursive formula to compute the subproblems : $T[1..n]$ is the profit in Toronto, $M[1..n]$ is the profit in Melbourne

1. $dp_T[i] = \max(dp_T[i-1], dp_M[i-2]) + T[i]$
2. $dp_M[i] = \max(dp_M[i-1], dp_T[i-2]) + M[i]$

where $[i-2]$ indicate the two day ago Emon is in another city, $[i-1]$ indicate that Emon is in same city yesterday.

Justification why (†) correctly computes (*): With this formula, If Emon is in Toronto on day i , we can consider two cases:

- case 1: he stays in Toronto, in this way he builds upon the maximum profit from the previous day when he was already in Toronto ($dp_T[i-1]$).

- case 2: he move to another city. Thus, spent one day traveling, and arrived in Toronto on day i . This scenario uses $dp_M[i-2]$. Assume for contradiction that this formula does not give the maximum profit. This would imply there is a higher profit configuration where Emon traveled from Melbourne. However, the constraints allow only traveling from Melbourne two days prior, as traveling consumes an entire day. If there were any other way to achieve higher profit, it would contradict the problem's rules, as only the previous scenarios (staying in Toronto or traveling from Melbourne) are feasible.

According to this, if day i emon is in melbourne, there will be same situation consider.

Running time : The algorithm totally pass n days and no-matter if-else or the return or the insert cost constant time, so the total running time is $\Theta(n)$

Algorithm 4 $\text{Do}(T, M)$

```
1:  $n := \text{length}[T]$ 
2:  $dp_T[1] = T[1]$ 
3:  $dp_M[1] = M[1]$ 
4: for  $i := 2$  to  $n$  do
5:    $dp_T[i] = \max(dp_T[i-1], dp_M[i-2]) + T[i]$ 
6:    $dp_M[i] = \max(dp_M[i-1], dp_T[i-2]) + M[i]$ 
7: end for
8: return  $\max(dp_T[n], dp_M[n])$ 
```
